TRW Auto-Trade Signal Bot

Automatically execute Prof Adam's RSPS portfolio signals on Hyperliquid

GitHub Repo: https://github.com/adamsgoldenbullrun2026/trw
Clone it: `git clone https://github.com/adamsgoldenbullrun2026/trw.git`

WORK IN PROGRESS: This bot was built today and is still in early development. It works, but there may be bugs. In particular:

- The TRW session token may expire and need refreshing — I'm not sure how long it lasts yet
- Edge cases in signal parsing may need fixing as new signal formats appear
- This will need maintenance going forward as TRW and Hyperliquid update their platforms

Use at your own risk. Start with a small amount to test. I'll keep everyone updated with fixes and improvements. If you find a bug, let me know.

Contents

1. How It Works
2. What You Need
3. Step 1: Get Your TRW Session Token
4. Step 2: Set Up Hyperliquid API
5. Step 3: Set Up Modal (Free Cloud Hosting)
6. Step 4: Set Up Slack Notifications (Optional)
7. Step 5: Install & Deploy
8. Step 6: Test Everything
9. Using the Dashboard
10. Maintenance
11. Trust & Verification

12. Security
13. Costs
14. FAQ

How It Works

```
Architecture:

TRW Signal Channel       Hyperliquid         Slack
     |                       ▲                   ▲
     ▼                       |                   |
┌──────────────┐      ┌────────────┐      ┌──────────┐
│ Read Signal  │ ──▶  │ Rebalance  │ ──▶  │ Notify   │
└──────────────┘      └────────────┘      └──────────┘
                ▲
                |
       Modal Cloud (cron)
       Runs 24/7 for free
```

The bot runs in the cloud on **Modal** (free tier). Every few minutes, it:

1. **Reads** the latest signal from Prof Adam's Portfolio Signals channel via TRW's API
2. **Parses** the allocation percentages (e.g., 80% ETH, 14.3% HYPE, 5.7% PAXG)
3. **Compares** to your current Hyperliquid positions
4. **Rebalances** if the signal has changed — using 1x leverage perps (spot-equivalent)
5. **Notifies** you on Slack with what it did

> **Polling schedule (UTC):**
> 00:00–00:30 → every 2 minutes (peak signal window)
> 00:30–05:00 → every 10 minutes
> 05:00–00:00 → every hour
>
> **Trading mode:**

> 00:00–05:00 → fully autonomous (auto-executes trades)
> 05:00–00:00 → sends you a Slack notification to approve before trading

What You Need

Item	Cost	Notes
TRW Subscription	You already have this	Investing Masterclass campus
Hyperliquid Account	Free	With USDC deposited for trading
Modal Account	Free	Cloud hosting — runs the bot 24/7
Slack Workspace	Free	For notifications (optional but recommended)
Python 3.10+	Free	For initial setup only — not needed after deploy

> **Total cost: $0/month** (beyond your existing TRW subscription and Hyperliquid trading fees of ~0.04%)

Step 1: Get Your TRW Session Token

The bot needs to read messages from the signal channel. TRW uses a session token for authentication.

1. Open **The Real World** in Chrome or Brave
2. Navigate to **Adam's Portfolio Signals** channel
3. Press **F12** to open DevTools
4. Click the **Network** tab
5. Click the **Fetch/XHR** filter button
6. Reload the page (**F5**)
7. Click on any request in the list
8. In the **Headers** tab, scroll down to **Request Headers**
9. Find `x-session-token` — copy the entire value

> **NEVER share your session token with anyone.** Copy it to your clipboard and use it ONLY in the manager GUI (`python manage.py`). Do not paste it into Discord, AI chats, websites, or anywhere else. The only places your token should ever exist are:
>
> - The manager GUI running on your local machine
> - Your local `.env` file (if you know what you're doing)
>
> If anyone asks you for your session token — for any reason — they are trying to scam you.

How long does it last? Session tokens appear to be long-lived (days to weeks). If the bot stops working, you may need to re-grab the token. The bot will notify you on Slack if this happens.

Step 2: Set Up Hyperliquid API

The bot needs an API wallet to place trades on your behalf. API wallets **cannot withdraw funds** — they can only place and cancel orders.

1. Go to **app.hyperliquid.xyz** and make sure your wallet is connected and you have funds deposited
2. Click **More** → **API** in the top menu
3. Give your API wallet a **name** (e.g., "signal-bot") — this is important because generating a new *unnamed* API wallet will deregister any previous unnamed one
4. Click **Generate**, then click **Authorize API Wallet**
5. Set the **Days Valid** field — you can go up to **180 days** max (recommended: set it to the maximum)
6. Copy and securely store the **Private Key** it gives you — this is what the bot uses to sign trades
7. Click **Authorize** and **sign the transaction in your wallet** (MetaMask will pop up — don't skip this step or the key won't work)
8. Grab your **Master Account Address** from the dropdown next to your connected wallet in the top right corner of the Hyperliquid app

You end up with three pieces of info:

- **API Wallet Address** — the public address of the generated wallet (you don't need this for the bot)
- **Private Key** — the secret key for signing trades (goes into your .env as `HYPERLIQUID_API_PRIVATE_KEY`)
- **Master Account Address** — your main wallet's public address (goes into your .env as `HYPERLIQUID_MASTER_ACCOUNT_ADDRESS`)

NEVER share these keys with anyone. Copy them to your clipboard and use them ONLY in the manager GUI (`python manage.py`). Do not paste them into Discord, AI chats, websites, or anywhere else. The only places your keys should ever exist are:

- The manager GUI running on your local machine
- Your local `.env` file (if you know what you're doing)

If anyone asks you for your private key or session token — for any reason — they are trying to scam you.

Key things to remember:

- The API wallet can place and cancel orders but **cannot withdraw funds**
- The bot uses your **master account address** to query your positions and balance, and the **API private key** to sign trades
- If you generate a new *unnamed* API wallet, it **deregisters the previous unnamed one** — always name your wallets if you want multiple
- The key expires after the validity period you set — set it to 180 days and put a reminder in your calendar to regenerate it

Deposit USDC

Make sure you have USDC in your **Perps wallet** on Hyperliquid (not Spot). You can transfer from Spot to Perps in the Portfolio page.

Step 3: Set Up Modal (Free Cloud Hosting)

Modal runs your bot 24/7 in the cloud for free. You don't need to leave your computer on.

Create a Modal account

1. Go to modal.com
2. Click **"Sign Up"** — you can use Google or GitHub to sign up
3. No credit card required — the free tier is more than enough for this bot

Install the Modal CLI

Open a terminal/command prompt and run:

```
pip install modal
```

Connect Modal to your account

Run this command and it will open a browser window to authenticate:

```
modal token new
```

Click **"Approve"** in the browser when prompted. You should see a success message in your terminal like:

```
Token verified successfully!
Token written to ~/.modal.toml
```

Verify it works

```
modal profile current
```

This should print your Modal workspace name (e.g., `your-username-1234`). If you see an error, try running `modal token new` again.

> **That's it!** Modal is now set up. The manager (`python manage.py`) will handle creating secrets and deploying the bot to your Modal account automatically.

Step 4: Set Up Slack Notifications (Optional)

Slack notifications let you know when the bot detects a signal and what trades it made. Recommended but not required.

1. Go to **api.slack.com/messaging/webhooks**
2. Click **"Create your Slack app"** → **From scratch**
3. Name it "Signal Bot", pick your workspace
4. Go to **Incoming Webhooks** → toggle **ON**
5. Click **"Add New Webhook to Workspace"**
6. Pick the channel you want notifications in
7. Copy the webhook URL

> **Free alternative:** If you don't want to use Slack, you can skip this step. The bot will still run, you just won't get notifications. You can always check the web dashboard instead.

Step 5: Install & Deploy

Option A: Automated Setup (Recommended)

1. Install Python 3.10+ from **python.org** if you don't have it
2. Open a terminal/command prompt
3. Download the bot code — pick one method:

   Option 1: Using git (if you have it installed)

   ```
   git clone https://github.com/adamsgoldenbullrun2026/trw.git
   cd trw
   ```

   Option 2: Download as ZIP (no git needed)

   a. Go to https://github.com/adamsgoldenbullrun2026/trw

   b. Click the green **"Code"** button → **"Download ZIP"**

   c. Extract the ZIP to a folder (e.g. your Desktop)

   d. Open a terminal in that folder:

   - **Windows:** Open the extracted folder in File Explorer, click the address bar at the top, type `cmd`, and press Enter
   - **Mac:** Right-click the folder → **"Open Terminal"** (or Services → New Terminal at Folder)

4. Install dependencies:

   ```
   pip install -r requirements.txt
   ```

5. Make sure you've completed **Step 3** (Modal is installed and `modal token new` has been run)
6. Launch the setup manager:

   ```
   python manage.py
   ```

7. A web page will open in your browser with a step-by-step form. Paste in your tokens from Steps 1, 2, and 4, then click **Save & Deploy**.

> **That's it!** The manager handles everything — saving your tokens, creating Modal secrets, and deploying the bot. After the first setup, run `python manage.py` again any time you need to update tokens, check status, or redeploy.

Option B: Using AI to Help (If You Get Stuck)

If you're not comfortable with the command line, you can use a free AI assistant to help:

1. Go to **claude.ai** (free tier) or **chatgpt.com**
2. Tell it: "*I'm setting up a Python project. I have these files [paste the README]. Can you walk me through the setup step by step on [Windows/Mac]?*"
3. The AI will give you exact commands to run
4. When you need to enter your tokens, enter them directly in the manager or terminal — **never paste tokens into the AI chat**

> **NEVER paste your private keys, session tokens, or wallet addresses into any AI chat, website, or Discord message.** These should only go into the manager GUI or your local `.env` file.

Option C: CLI Setup (Alternative)

If you prefer the terminal over the GUI, you can use the command-line wizard instead:

```
python setup.py
```

Or to update individual tokens later:

```
python setup.py --reconfigure
```

Option D: Manual Setup

1. Copy `.env.example` to `.env`
2. Edit `.env` and fill in your actual values
3. Create the Modal secret:

```
modal secret create signal-bot-secrets \
  "TRW_SESSION_TOKEN=your_token" \
```

```
  "TRW_SIGNAL_CHANNEL_ID=01H83QAX979K9R7QTMH74ATR8C" \
  "TRW_PROF_ADAM_USER_ID=01GHHHWZE7Q77AKGWZDGC5PDCN" \
  "HYPERLIQUID_API_PRIVATE_KEY=0xyour_key" \
  "HYPERLIQUID_MASTER_ACCOUNT_ADDRESS=0xyour_address" \
  "SLACK_WEBHOOK_URL=your_webhook"
```

4. Deploy:

```
modal deploy modal_signal_bot.py
```

Step 6: Test Everything

1. **Test the signal reader:**

   ```
   python trw_signal_reader.py
   ```

   You should see the latest signal with allocations.

2. **Test Hyperliquid connection:**

   ```
   python hyperliquid_rebalancer.py --status
   ```

   You should see your account value and positions.

3. **Test the dashboard:**
   Open your Modal dashboard URL in a browser. It should show your positions and the latest signal.

4. **Test a preview (no actual trades):**

   ```
   python hyperliquid_rebalancer.py --preview-live
   ```

   This shows what trades WOULD be made without executing them.

> **If all tests pass, you're done!** The bot is running in the cloud and will automatically execute signals tonight.

Using the Dashboard

You have two dashboards:

Cloud Dashboard (for your phone)

This is hosted on Modal and accessible from anywhere — bookmark it on your phone.

URL format: `https://YOUR_WORKSPACE--signal-bot-web.modal.run`

The manager (`python manage.py`) shows your dashboard URL at the top of the page. You can also find it in Modal's web interface.

Features:

- View your current positions, PnL, and account value
- See the latest signal from Prof Adam
- Approve or dismiss pending signals (during non-autonomous hours)
- Force a rebalance manually
- Run health checks

Local Manager (for your computer)

```
python manage.py
```

Use this to update tokens, check connection status, and redeploy. Only runs on your local machine.

Modal Web Interface

You can also monitor your bot directly on Modal's website:

- Go to modal.com and sign in
- Click on the **signal-bot** app to see logs, run history, and errors
- This is useful for debugging if something goes wrong

Maintenance & Updating Tokens

Made a mistake during setup? Token expired? Want to add Slack later? There are two easy ways to update your settings:

Option 1: GUI Manager (Recommended)

Run this in the bot folder:

```
python manage.py
```

This opens a local web page in your browser where you can:

- See the current status of all connections (TRW, Hyperliquid, Slack)
- Update any token or key by pasting in the new value
- Save and redeploy with one click

> **This runs locally on your computer** — it's not accessible from the internet. Your tokens are shown on the page so you can verify they're correct, but they never leave your machine except to the services you configured.

Option 2: Command Line

```
# Update specific tokens interactively
python setup.py --reconfigure

# Just redeploy with current settings
python setup.py --redeploy
```

The `--reconfigure` command gives you a menu to update just the TRW token, Hyperliquid keys, or Slack webhook individually.

TRW Token Expired

If the bot stops reading signals or you get a Slack notification about auth failure:

1. Open TRW in your browser, go to the signal channel
2. F12 → Network → Fetch/XHR → reload → grab new `x-session-token`

3. Run `python manage.py` → paste the new token → click **Save & Deploy**

Or from the command line: `python setup.py --reconfigure` → choose option 1.

Hyperliquid API Key Expired

API keys expire after the validity period you set (max 180 days).

1. Generate a new key at **app.hyperliquid.xyz** → **More** → **API**
2. Run `python manage.py` → paste the new key → click **Save & Deploy**

Or: `python setup.py --reconfigure` → choose option 2.

Adding Slack Later

If you skipped Slack during setup and want to add it now:

1. Create a Slack webhook (see Step 4 above)
2. Run `python manage.py` → paste the webhook URL → click **Save & Deploy**

Updating the Bot

If you download an updated version of the code, run `python setup.py --redeploy` or use the manager.

Trust & Verification — "How Do I Know This Is Safe?"

You're right to be cautious. You're about to enter API keys for your trading account into code some stranger wrote. Here's why it's safe, and how you can prove it to yourself.

The code is 100% open source and auditable

There are only 5 small Python files, totalling ~600 lines of actual logic. This isn't a compiled binary or obfuscated mess — it's plain, readable Python. Every single line is visible to you.

The code only talks to 3 services — all yours

Service	URL	Why
TRW	`eden.therealworld.ag`	Read signals from the chat (same server your browser uses)
Hyperliquid	`api.hyperliquid.xyz`	Read positions & place trades (Hyperliquid's official API)
Slack	Your own webhook URL	Send YOU notifications (a URL YOU created)

That's it. No other HTTP calls exist. No data goes to the author or any third party. Zero analytics, zero telemetry, zero phone-home.

It runs on YOUR infrastructure

The bot runs on YOUR Modal account that YOU created. Your secrets are encrypted in YOUR Modal vault. The author has zero access to your Modal account, your dashboard, your keys, or your trades.

How to verify it yourself

> **Option 1 (10 seconds):** Search each file for `requests.post` and `req.post`. Every single network call goes to one of the 3 URLs listed above. There are no hidden calls.
>
> **Option 2 (30 seconds):** Copy the entire contents of any file and paste it into **ChatGPT** or **Claude** (the free versions work fine). Ask:

> *"Does this code send my credentials, private keys, or personal data anywhere other than eden.therealworld.ag, api.hyperliquid.xyz, and my own Slack webhook?"*
> The answer will be no.
>
> **Option 3 (for the paranoid):** Fork the repo, read every line yourself, and run your own copy. The code is intentionally simple and heavily commented so you can understand exactly what it does.

What the API wallet CAN'T do

Even in the worst-case scenario where someone somehow gets your Hyperliquid API key:

- They **cannot withdraw** your funds — withdrawals require your master wallet
- They **cannot transfer** funds between wallets
- They can only place and cancel orders — and you can **instantly revoke** the API key at app.hyperliquid.xyz to stop them

Security

What the API wallet can do

- Place and cancel orders ✓
- Withdraw funds ✗ (impossible)
- Transfer between wallets ✗ (impossible)

Even if your API key is compromised, your funds cannot be stolen. The worst case is someone opens bad positions — which you can close immediately by revoking the API wallet at app.hyperliquid.xyz.

Where your secrets are stored

- **Local .env file** — on your computer only, never committed to Git
- **Modal encrypted secrets** — encrypted at rest, only accessible during function execution
- **Nowhere else** — not in the code, not in logs, not in the dashboard

> **Rules:**
>
> - Never share your .env file

- Never paste tokens into AI chats, Discord, or any website
- Never commit .env to Git (the .gitignore prevents this)
- If you suspect a key is compromised, revoke the Hyperliquid API wallet immediately

Costs

Service	Cost
Modal (cloud hosting)	Free (generous free tier covers this easily)
Hyperliquid trading fees	~0.04% taker / ~0.01% maker per trade
Hyperliquid funding rates	Variable, usually small (positive or negative)
Slack	Free
Total monthly	**~$0 + trading fees**

FAQ

Why perps instead of spot?

ETH, PAXG, and XAUT don't have spot markets on Hyperliquid — they only exist as perps. Using 1x leverage perps gives identical economic exposure to holding spot. All positions use 1x cross leverage (no amplified risk).

What if the signal says "No change"?

The bot detects this and does nothing. It sends a Slack notification confirming "No change" so you know it's working.

What if the professor posts other messages (not signals)?

The bot filters for messages containing "Portfolio Signal Update" from Prof Adam specifically. It ignores everything else.

What if my internet goes down?

The bot runs in Modal's cloud, not on your computer. Your internet doesn't matter once deployed.

What if Hyperliquid is down?

The bot will fail to execute trades and send you a Slack error notification. It will retry on the next poll cycle.

Can I stop the bot temporarily?

Yes: `modal app stop signal-bot`
Restart: `modal deploy modal_signal_bot.py`

Can I change the schedule?

Yes — edit the `should_poll_now()` function in `modal_signal_bot.py` and redeploy.

Can I use this for other signal channels?

Yes — change `TRW_SIGNAL_CHANNEL_ID` and `TRW_PROF_ADAM_USER_ID` in your .env to match a different channel/professor. You may need to adjust the signal parser if the format is different.

> **Disclaimer:** This bot was built in a single day and is a work in progress. It functions but may contain bugs. The TRW session token expiry period is unknown and may require periodic refreshing. The signal parser handles all currently known signal formats but may need updating if Prof Adam changes the format. Start with a small amount to test. I'll post updates and fixes as issues are discovered. Use at your own risk — this is not financial advice. Always understand the signals before automating them.

Built for the TRW Investing Masterclass community.